

TEORETICKÁ INFORMATIKA

Dijsktrův algoritmus

David Weber
weber3@spsejecna.cz



Rok 2025/26

Co již známe...

- Již známe algoritmy BFS a DFS.
- Algoritmus BFS nalezne vždy nejkratší cestu do všech **dosažitelných vrcholů** v **neohodnoceném grafu**.
- Časová i prostorová složitost činí $O(n + m)$.

Co když budeme hledat cestu v **ohodnoceném grafu**?

Ohodnocené grafy

Ohodnocený graf

Definice (Ohodnocený graf)

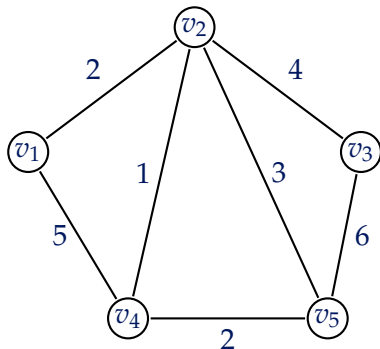
Ohodnocený grafem rozumíme uspořádanou trojici (V, E, w) , přičemž $G = (V, E)$ je libovolný graf a $w : E \rightarrow \mathbb{R}$ je zobrazení přiřazující každé hraně $e \in E$ její **váhu** $w(e)$.

Poznámka

Někdy je výhodnější se odkázat na samotné konceptvé vrcholy hrany u, v . Abychom si usnadnili zápis, budeme psát $w(u, v)$ místo $w(\{u, v\})$ v případě **neorientovaného grafu**, resp. místo $w((u, v))$ v případě **orientovaného grafu**.

Ohodnocený graf

Příklad



$$w(\{v_1, v_2\}) = w(v_1, v_2) = 2,$$

$$w(\{v_2, v_3\}) = w(v_2, v_3) = 4,$$

$$w(\{v_1, v_4\}) = w(v_1, v_4) = 5,$$

$$w(\{v_2, v_4\}) = w(v_2, v_4) = 1,$$

$$w(\{v_2, v_5\}) = w(v_2, v_5) = 3,$$

$$w(\{v_3, v_5\}) = w(v_3, v_5) = 6,$$

$$w(\{v_4, v_5\}) = w(v_4, v_5) = 2$$

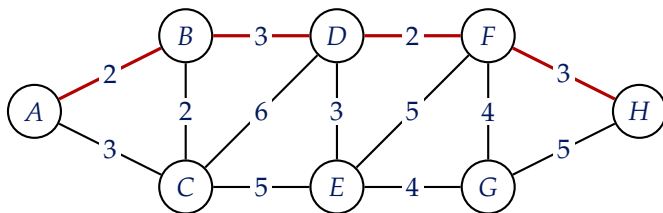
Vzdálenost vrcholů v ohodnoceném grafu

Definice

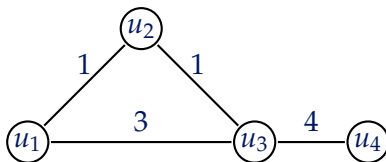
Nechť $G = (V, E, w)$ je ohodnocený graf, vrcholy $u, v \in V$. Pak vzdáleností vrcholů u, v rozumíme hodnotu

$$d(u, v) = \min \left\{ \sum_{e \in P, e \in E} w(e) : P \text{ je cesta z } u \text{ do } v \right\}.$$

Pokud v není z u dosažitelný, pak definujeme $d(u, v) = \infty$.

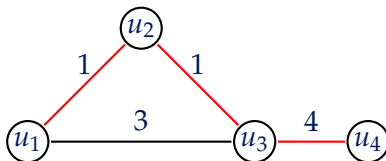


Hledání cest v ohodnocených grafech



Jaká je nejkratší cesta z u_1 do u_4 ?

- BFS by našel cestu (u_1, u_3, u_4) .
- Nicméně nejkratší cesta je (u_1, u_2, u_3, u_4) , **přestože má více hran!**



Dijkstrův algoritmus

Dijkstrův algoritmus

- Egster Dijkstra (1958)
- Na vstupu máme **nezáporně** ohodnocený graf $G = (V, E, w)$ a počáteční vrchol $v_0 \in V$.

Zatím není zcela jasné, proč chceme, aby váhy hran byly **nezáporné**, ale později si to zdůvodníme.

- Stejně jako v případě BFS i zde bychom rádi **při uzavírání vrcholu** chtěli znát nejkratší cestu do něj.
- Nicméně již nemusí platit, že **první nalezená cesta** od libovolného vrcholu v je nutně nejkratší.

Cestu do v je třeba postupně **zlepšovat!**

- Stejně jako v případě BFS si budeme uchovávat **stavy** a **vzdáleností** vrcholů.

Postup Dijkstrova algoritmu

- 1 Inicializujeme seznamy stavů a vzdáleností vrcholů (vrcholy budou z počátku nenalezené a jejich vzdálenosti ∞).
- 2 Označíme počáteční vrchol v_0 jako **otevřený** a nastavíme $D(v_0) = 0$.
- 3 Vybereme otevřený vrchol (pokud nějaký existuje) v s nejmenší hodnotou $D(v)$.
- 4 Vezmeme všechny sousedy s vrcholu v a pokud z některého vede kratší cesta do v , **aktualizujeme** $D(v)$ a s označíme jako **otevřený**.
- 5 **Uzavřeme** vrchol v , vrátíme ke 3. kroku a postup opakujeme.

Pseudokód Dijkstrova algoritmu

Algoritmus: DIJKSTRA

Vstup: Nezáporně ohodnocený graf $G = (V, E, w)$
a počáteční vrchol $v_0 \in V$

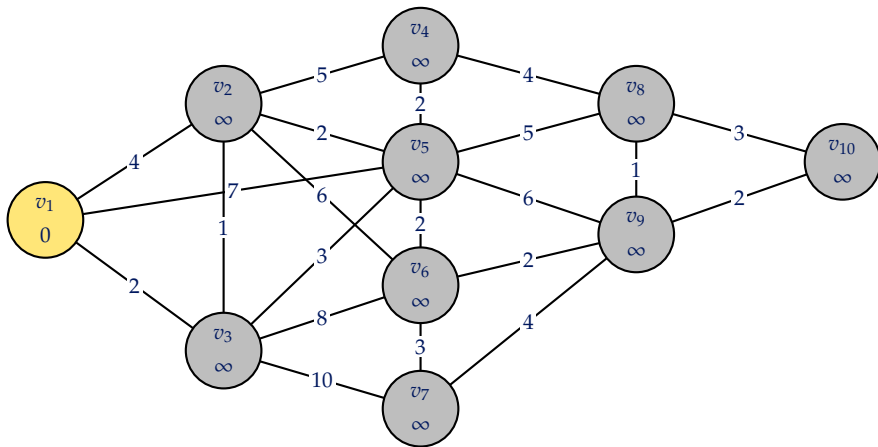
```
1 foreach vrchol  $v \in V$  do
2    $stav(v) \leftarrow \textit{nenalezený}, D(v) \leftarrow \infty$ 
3  $stav(v_0) \leftarrow \textit{otevřený}, D(v_0) \leftarrow 0$ 
4 while existují otevřené vrcholy do
5    $v \leftarrow$  otevřený vrchol s nejmenším  $D$ 
6   foreach soused  $s \in V$  vrcholu  $v$  do
7     if  $D(v) + w(v, s) < D(s)$  then
8        $D(s) \leftarrow D(v) + w(v, s)$ 
9        $stav(s) \leftarrow \textit{otevřený}$ 
10   $stav(v) \leftarrow \textit{uzavřený}$ 
```

Výstup: Seznam vzdáleností D

Dijkstrův algoritmus

Příklad

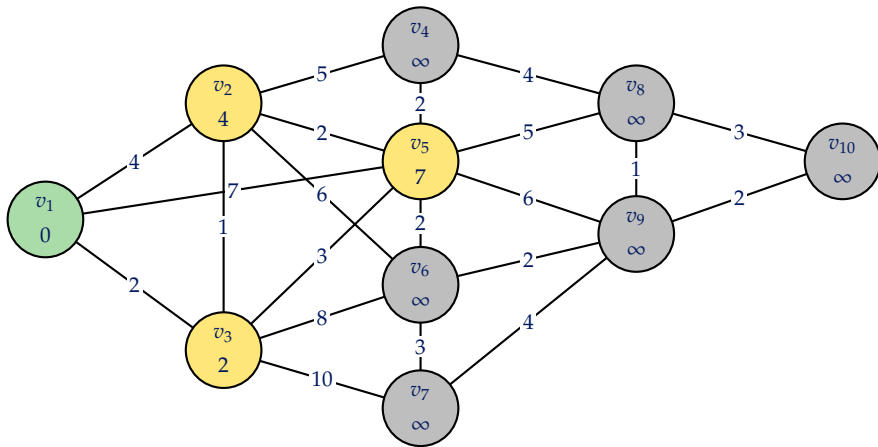
Krok 0: inicializace



Dijkstrův algoritmus

Příklad

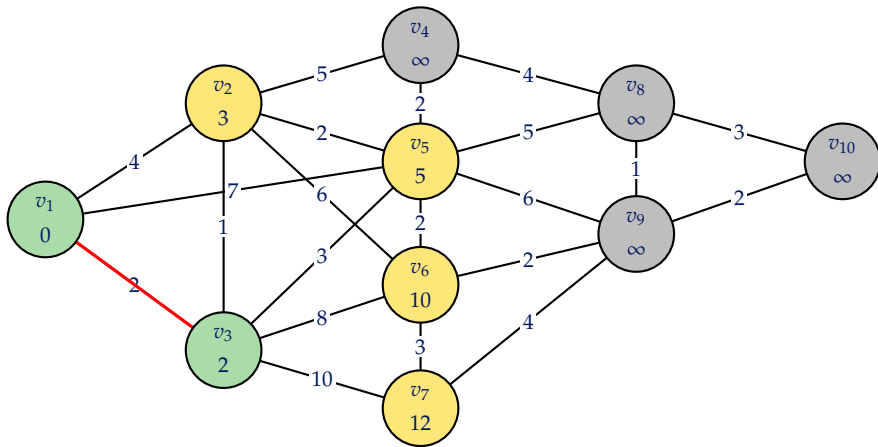
Krok 1: uzavíráme v_1



Dijkstrův algoritmus

Příklad

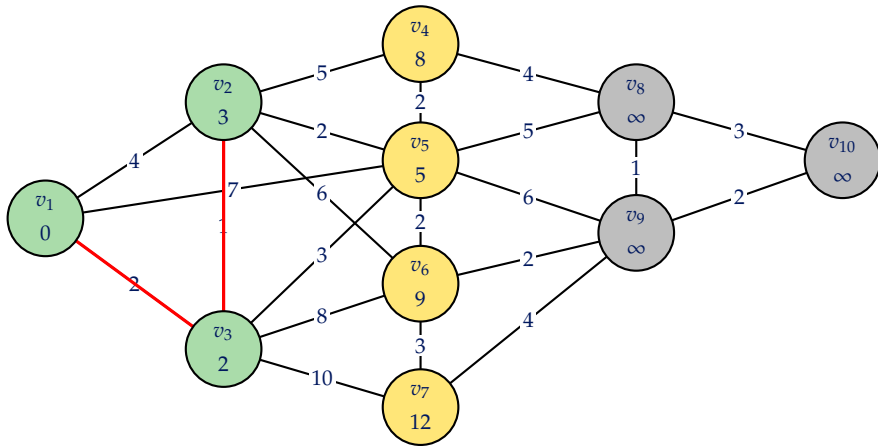
Krok 2: uzavíráme v_3



Dijkstrův algoritmus

Příklad

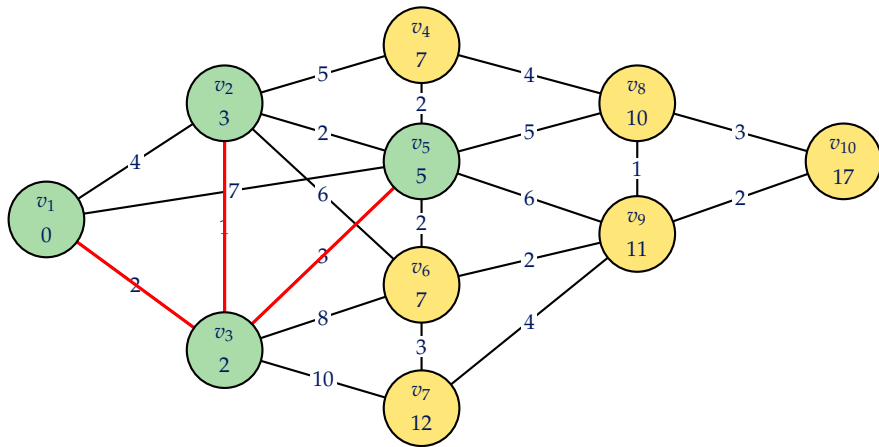
Krok 3: uzavíráme v_2



Dijkstrův algoritmus

Příklad

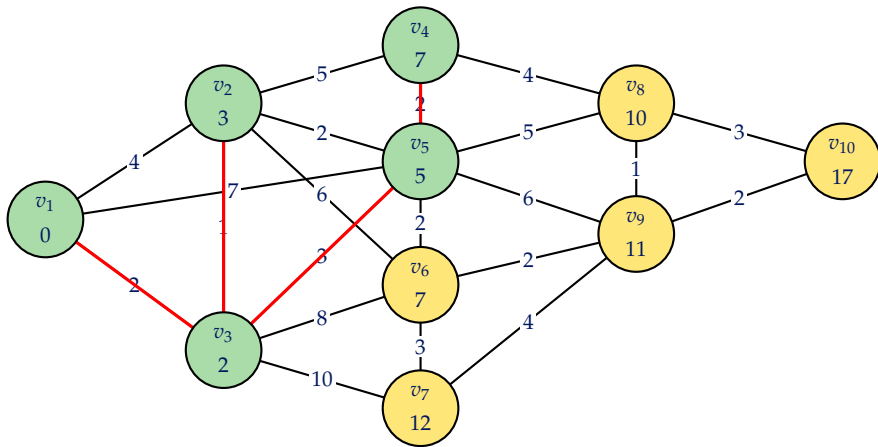
Krok 4: uzavíráme v_5



Dijkstrův algoritmus

Příklad

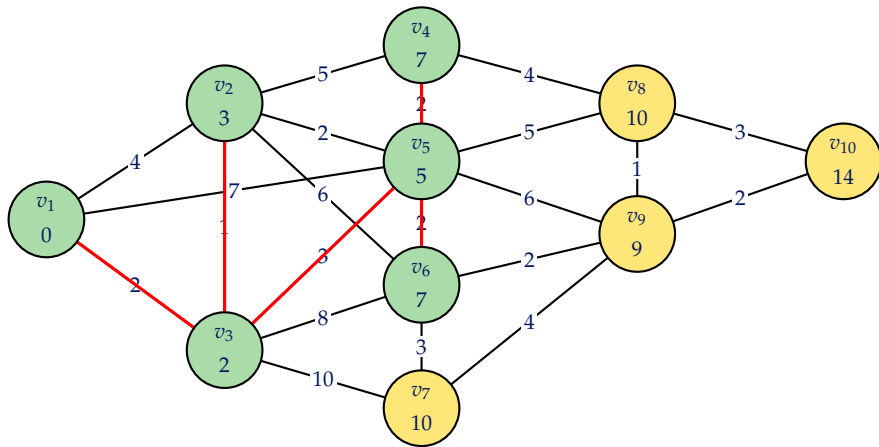
Krok 5: uzavíráme v_4



Dijkstrův algoritmus

Příklad

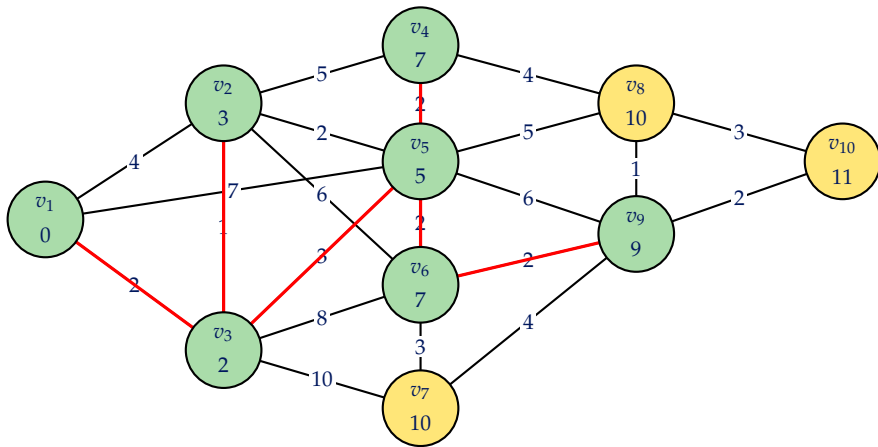
Krok 6: uzavíráme v_6



Dijkstrův algoritmus

Příklad

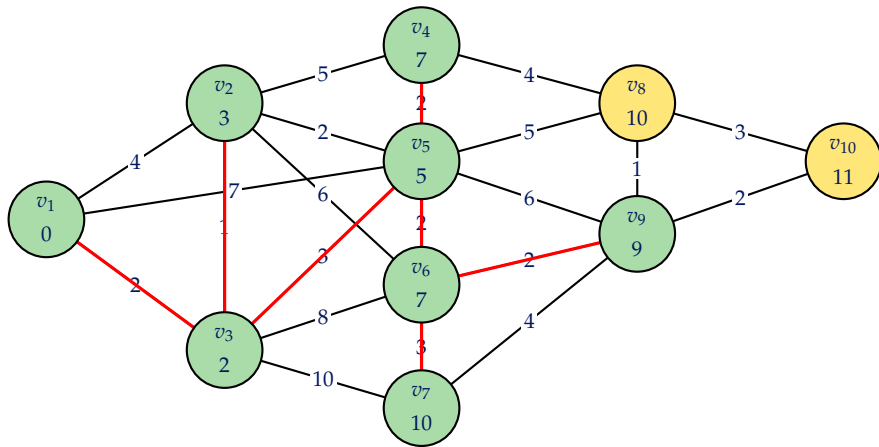
Krok 7: uzavíráme v_9



Dijkstrův algoritmus

Příklad

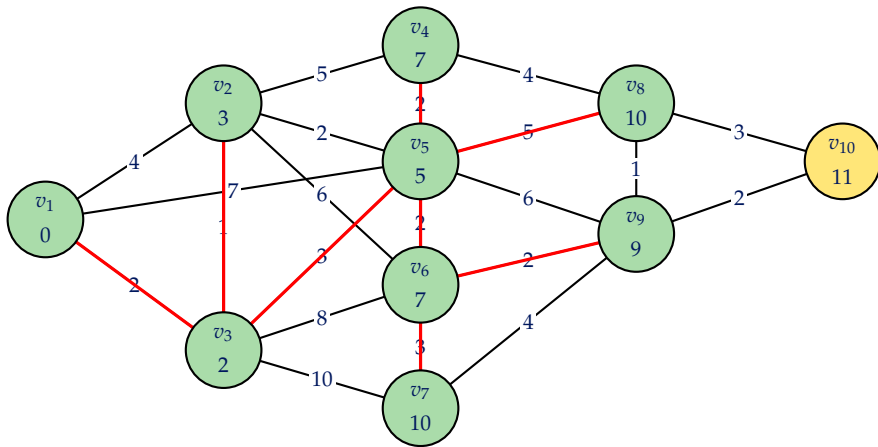
Krok 8: uzavíráme v_7



Dijkstrův algoritmus

Příklad

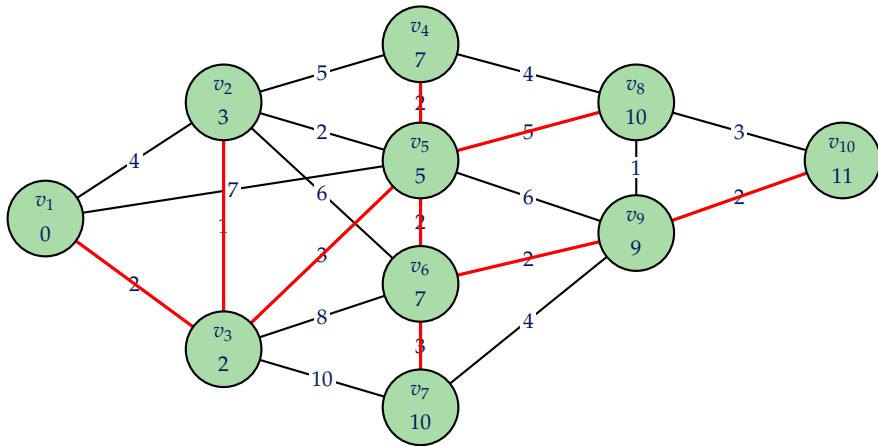
Krok 9: uzavíráme v_8



Dijkstrův algoritmus

Příklad

Krok 10: uzavíráme v_{10} – konec



Proč to funguje?

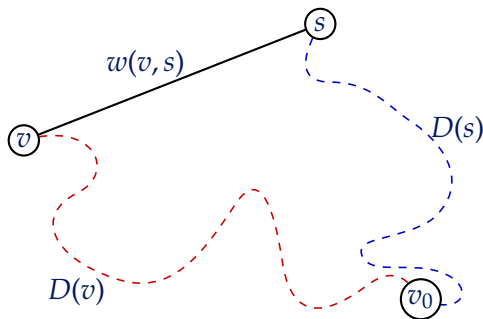
Tvrzení

Když Dijkstrův algoritmus otevírá vrchol v , pak platí, že $D(v) = d(v, v_0)$.

- Jinak řečeno, když si algoritmus vybere vrchol v , aby jej otevřel, jeho vzdálenost od počátku je **spočítaná správně**.
- Na tuto skutečnost můžeme nahlédnout sporem.

Proč to funguje?

Co by se stalo, kdyby algoritmus vybral vrchol v , jehož D neodpovídá délce nejkratší cesty?



- $D(v)$ = spočítaná cesta do v .
- $D(s) + w(v, s)$ = nejkratší cesta do v vedoucí přes souseda s .

Proč to funguje?

- Protože $D(v)$ je větší než délka nejkratší cesty do v , platí

$$D(v) > D(s) + w(v, s).$$

- Připomeňme si nyní, že váhy všech hran jsou nezáporné.
- Pak tedy platí:

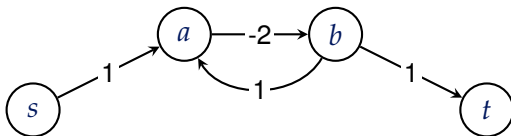
$$D(v) > D(s) + \underbrace{w(v, s)}_{\geq 0} \geq D(s).$$

- To znamená, že $D(s)$ je **nižší než** $D(v)$.
- **To je ale spor**, neboť algoritmus by si tedy **nevybral** vrchol v .

Nezápornost vah

Všimněte si, že právě díky **nezápornosti vah** hran grafu na vstupu algoritmus funguje!

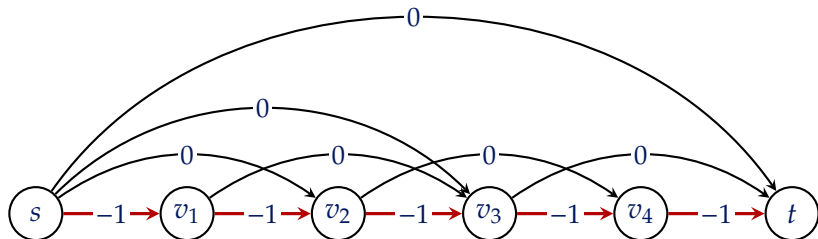
Kdyby hrany mohly mít i záporné váhy, nejkratší sled mezi některými dvojicemi vrcholů vůbec nemusí existovat!



$$d(s, t) = -\infty$$

Nezápornost vah

Dále se také může stát, že hledání nejkratší cesty přejde na problém hledání **Hamiltonovské cesty** $\Rightarrow$ NP-úplný problém.



Nejkratší cesta z s do t je

$s \rightarrow v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_4 \rightarrow t$ s cenou -5

Složitost algoritmu

Kam ukládat vrcholy?

- Zatím jsme vůbec nevyřešili, jak budeme ukládat hodnoty D .
- Jako první varianta se nabízí **obyčejný seznam/pole**.

Zkusíme určit **časovou** a **prostorovou složitost**.

Časová složitost s polem

- Opět si označíme $n = |V|$ a $m = |E|$.
- Inicializace potvrzuje $O(2n) = O(n)$, neboť zavádíme seznam stavů a vzdáleností **pro všechny vrcholy**.
- Každý vrchol uzavřeme **nejvýše jednou** $\Rightarrow$ vnější cyklus provede nejvýše $O(n)$ iterací.
- Hledáním minimální hodnoty D strávíme $O(n) \Leftarrow$ **hledáme minimum v poli**.
- Každou hranu při procházení sousedů projdeme **nejvýše dvakrát** $\Rightarrow$ to dává řádově $O(2m) = O(m)$ kroků.
- Aktualizaci hodnoty $D(v)$ libovolného vrcholu v provedeme v konstantním čase.
- Dohromady:

$$O(\underbrace{n}_{\text{inicializace}} + \underbrace{n}_{\text{vnejší cyklus}} \cdot \underbrace{n}_{\text{minimum}} + \underbrace{m}_{\text{sousedé}}) = O(n^2 + m).$$

Prostorová složitost s polem

- Pokud budeme reprezentovat graf pomocí **seznamu sousedů**, spotřebujeme $O(n + m)$ paměti.
- Dále již potřebujeme jen pole pro uchování stavů a vzdáleností.
- Celkově činí prostorová složitost $O(n + m)$.

S polem jsme dosáhli následujících výsledků:

- Časová složitost: $O(n^2 + m)$.
- Prostorová složitost $O(n + m)$.

S prostorovou složitostí toho moc nezmůžeme, nicméně v případě časové složitosti **lze výsledek o něco zlepšit** $\Rightarrow$ máme přeci **binární haldu**!

Ta se nám bude hodit v případě **výběru minima** z hodnot D .

Časová složitost s binární haldou

- Z počátku spotřebujeme $O(n)$ času na zavedení pole stavů.
- V případě haldy provedeme n -krát operaci **INSERT**, tzn. spotřebujeme $O(n \log n)$ času.
- Při výběru vrcholu s nejnižším D provádíme operaci **EXTRACTMIN**. Celkově ji provedeme n -krát (vnější cyklus), tzn. spotřebujeme $O(n \log n)$ času.
- Při aktualizaci hodnoty $D(v)$ libovolného vrcholu provádíme operaci **DECREASE**. Nejvýše jej provedeme m -krát (za každou hranu) $\Rightarrow$ spotřebujeme $O(m \log n)$.
- Dohromady:

$$\begin{aligned} O(\underbrace{n \log n}_{\text{inicializace}} + \underbrace{n}_{\text{vnější cyklus}} \cdot \underbrace{\log n}_{\text{EXTRACTMIN}} + \underbrace{m}_{\text{hrany}} \cdot \underbrace{\log n}_{\text{DECREASE}}) \\ = O((n + m) \log n). \end{aligned}$$

Shrnutí a závěrečný rozbor

Časová složitost **s polem** vs. **s haldou**:

- Dijkstrův algoritmus s polem: $O(n^2 + m)$
- Dijkstrův algoritmus s binární haldou: $O((n + m) \log n)$

Nyní se nabízí dvojice případů:

- Graf G je řídký, tj. má **nejvýše lineárně mnoho** hran vzhledem k počtu vrcholů, tj. $m = O(n)$. Pak
 - S polem: $O(n^2 + m) = O(n^2 + n) = O(n^2)$.
 - S haldou: $O((n + m) \log n) = O(2n \log n) = O(n \log n)$
- Graf G je hustý, tj. má **kvadraticky mnoho** hran vzhledem k počtu vrcholů, tj. $m = O(n^2)$. Pak
 - S polem: $O(n^2 + n^2) = O(2n^2) = O(n^2)$.
 - S haldou: $O((n + n^2) \log n) = O(n \log n + n^2 \log n) = O(n^2 \log n)$.

- Pro **řídke** grafy vychází asymptotická složitost lépe v případě **implementace s binární haldou**.
- Pro **husté** grafy vychází lépe **implementace s polem**.

- MAREŠ, Martin a VALLA, Tomáš. *Průvodce labyrintem algoritmů*. Druhé vydání. CZ.NIC. Praha: CZ.NIC, z.s.p.o., 2022. ISBN 978-80-88168-63-8.
- CORMEN, Thomas H.; LEISERSON, Charles Eric; RIVEST, Ronald L. a STEIN, Clifford. *Introduction to algorithms*. Fourth edition. Cambridge: The MIT Press, 2022. ISBN 978-0-262-04630-5.
- MATOUŠEK, Jiří a NEŠETŘIL, Jaroslav. *Kapitoly z diskrétní matematiky*. 4., upr. a dopl. vyd. V Praze: Karolinum, 2009. ISBN 978-80-246-1740-4.